Swarm and Evolutionary Computation 49 (2019) 62–74

Contents lists available at ScienceDirect

Swarm and Evolutionary Computation

journal homepage: www.elsevier.com/locate/swevo

Particle swarm optimization of deep neural networks architectures for image classification

Francisco Erivaldo Fernandes Junior, Gary G. Yen *

School of Electrical and Computer Engineering, Oklahoma State University, Stillwater, OK, 74078, USA

ARTICLE INFO

Keywords:
Particle swarm optimization
Deep neural networks
Convolutional neural networks
Image classification

ABSTRACT

Deep neural networks have been shown to outperform classical machine learning algorithms in solving real-world problems. However, the most successful deep neural networks were handcrafted from scratch taking the problem domain knowledge into consideration. This approach often consumes very significant time and computational resources. In this work, we propose a novel algorithm based on particle swarm optimization (PSO), capable of fast convergence when compared with others evolutionary approaches, to automatically search for meaningful deep convolutional neural networks (CNNs) architectures for image classification tasks, named psoCNN. A novel directly encoding strategy and a velocity operator were devised allowing the optimization use of PSO with CNNs. Our experimental results show that psoCNN can quickly find good CNN architectures that achieve quality performance comparable to the state-of-the-art designs.

1. Introduction

In recent years, deep neural networks (DNNs) have become the gold standard algorithm in the field of computer vision. More specifically, deep convolutional neural networks (CNNs) can obtain the best results in almost all image classification benchmarks, surpassing the classification capabilities of human experts [1,2]. However, it remains non-trivial to design a meaningful CNN architecture. For example, VGG16 [3], Inception [4], ResNet [5], and DenseNet [6], some of the most successful CNNs, were carefully handcrafted by taking into consideration the characteristics of the problem domain knowledge.

The authors of the VGG16 neural network were the first ones to show that deeper networks with small convolutional filters could achieve better results than those of shallow networks. The Inception's authors showed that the concept of network-in-network [7], where each layer of the neural network is another network itself, could be used to build networks deeper while being computationally efficient. ResNets use shortcut connections, also called skip or residual connections, to connect a layer's inputs to its outputs. Instead of computing a mapping between inputs and outputs, ResNets compute the residual mapping which makes the training procedure easier. On the other hand, DenseNets connect the outputs of each layer to every subsequent layer resembling the connectivity pattern of fully-connected neural networks. The connectivity pattern used by ResNet and DenseNet avoids the problem of vanishing gradient allowing deeper networks with thousand of layers. However, these and many others improvements in CNN architectures were achieved by trial and error. Thus, the creation of better CNNs remains a heuristic process that needs many insights about the specific problem domain, and, therefore, no structured means of designing them has been devised.

However, one can take inspiration from nature. The structure of animals' brains has evolved during billions of years through the mechanism of natural selection. Such approach is equally applicable in the field of Artificial Neural Networks (ANNs), and it is known as *NeuroEvolution*. This approach allows us to evolve either ANNs' architectures and weights. In its early inception, in the 1990s, *NeuroEvolution* was used specifically to update weights in a fixed ANN architecture avoiding many of the shortcomings produced by backpropagation. Back then, researchers did not have access to large enough data to train ANNs with backpropagation which frequently produced overfitting and local minima problems [8–10]. However, training ANNs with evolutionary algorithms has one major drawback: it takes longer to find a good set of weights than with backpropagation [11]. Thus, soon after the introduction of evolutionary algorithms for training fixed architectures ANNs, researchers began to use such algorithms to evolve weights and architectures at the same time. Such algorithms are known as Topology

* Corresponding author.
E-mail addresses: feferna@okstate.edu (F.E. Fernandes Junior), gyen@okstate.edu (G.G. Yen).

https://doi.org/10.1016/j.swevo.2019.05.010
Received 9 November 2018; Received in revised form 25 May 2019; Accepted 26 May 2019
Available online 1 June 2019

and Weight Evolving Artificial Neural Networks (TWEANNs). The NeuroEvolution of Augmenting Topologies (NEAT) created by Stanley and Miikkulainen in 2002 [12] is one of the most popular TWEANNs. NEAT starts with a basic single layer neural network and evolves it into more complex networks. NEAT is also capable of creating recurrent neural networks and avoids premature convergence by keeping a diverse number of neural networks through a speciation mechanism. The Evolutionary Acquisition of Neural Topologies (EANT) created by Siebel and Sommer in 2005 [13] is another example of TWEANN. Similar to NEAT, EANT starts with simple ANN architectures that grow more complex with each iteration of the algorithm. It contains two inner loops: one called *structural exploration* that tests new architectures through mutation, and another called *structural exploitation* that optimize weights using evolution strategy.

Due to the highly dimensional nature of neural networks, *NeuroEvolution*, however, has only been applied to shallow networks used mostly in reinforcement learning. For example, NEAT directly encodes ANNs which is not suitable for very complex networks because the computation quickly becomes intractable. Stanley et al. [14] tried to solve this problem with the Hypercube-Based NeuroEvolution of Augmenting Topologies (HyperNEAT) which makes use of connective Compositional Pattern Producing Networks (connective CPPNs) together with NEAT to indirectly encode neural networks. HyperNEAT takes into consideration the problem geometry when evolving neural networks, and, as an indirect encoding method, it can produce networks with millions of connections. However, neural networks produced by HyperNEAT cannot match the best CNN architectures produced by human experts. As a result, it is often considered better to use HyperNEAT as a feature extractor to other machine learning algorithms [15]. In 2017, researchers from Google developed the Large-Scale Evolution of Images Classifiers (LSEIC) algorithm which was able to overcome the limitations of traditional *NeuroEvolution* methods, and obtained state-of-the-art results in many benchmark datasets used with DNNs [16]. Liu et al. was also able to evolve DNNs achieving state-of-the-art results when compared with human-designed ones using hierarchical representations and mutation only [17]. Jin et al. developed *AutoKeras* in 2018 [18] which is also capable of searching for CNN architectures with state-of-the-art results. More recently, Sun et al. [19,20] developed the Evolving Deep Convolutional Neural Network (evoCNN), and the Evolving Unsupervised Deep Neural Network (EUDNN) algorithms to overcome the limitations of HyperNEAT. On evoCNN, a genetic algorithm is used to search for CNN architectures with specific crossover and mutation operators. While on EUDNN, a set of basis vectors is used to encode weights and connections efficiently. However, all those algorithms require huge amounts of computational resources that many researchers from other fields may not have access.

Particle Swarm Optimization (PSO) is another nature-inspired algorithm that can be used in the search for optimal neural networks architectures. Similarly to genetic algorithms, one can use PSO to evolve weights and architectures of neural networks. One of the earliest works using PSO to train an ANN was developed by Gudise and Venayagamoorthy in 2003 [21]. They showed that ANNs trained faster with PSO than with traditional backpropagation. Similarly, in 2007, Carvalho and Ludermir [22,23] developed two different PSO algorithms to both search for better architectures and train ANNs. They showed that PSO could also be used to improve ANN architectures producing competitive results when compared with other methods. However, the PSO algorithm developed therein and other works [24–27] can only search for optimal architectures of fully connected neural networks which are not suitable for image classification tasks. To overcome this limitation, Sun et al. [28] developed a PSO-based algorithm to automatically construct convolutional autoencoders (CAEs) and obtained state-of-the-art performance in multiples image classification datasets. Nevertheless, to the best of our knowledge, the work done by Wang et al. was the first one to develop an algorithm to directly evolve CNN architectures based on PSO, called IPPSO [29]. In their work, the particle encoding is inspired by computer networks, where each layer is assigned an IP address allowing the use of standard PSO. However, their algorithm can only deal with particles up to a preset maximum length, and their results are severely limited to only three different image classification datasets.

The main objective of this paper is to address the problem of searching for CNN architectures for image classification with a good balance between searching speed and classification accuracy. Therefore, we present our own implementation of PSO to address such a problem, which is called here as psoCNN. Our main contributions are the following:

1. A novel PSO algorithm is proposed that can search for optimal architectures in deep convolutional neural networks using variable-length particles with virtually no size limitation. Particles are allowed to grow in size without an upper bound.
2. A novel difference operator is presented that allows two particles with a different number of layers and parameters to be compared and allows particles' velocity to be updated without using real-valued encoding schemes.
3. A novel velocity operator is devised that can be used to modify a given CNN architecture to resemble the global best individual in the swarm or its personal best configuration. This velocity operator allows us to employ an almost standard PSO algorithm for searching, avoiding the use of multi-dimensional PSO algorithms.
4. A faster algorithm to find meaningful CNN architectures than previously available ones. PSO has been shown to converge quickly than genetic algorithms. By combining its fast convergence with the ability to search for CNN architectures, the proposed algorithm can exceed state-of-the-art results taking less time than competing algorithms.

The remainder of this paper is organized as follows: a detailed background and motivation about PSO and CNNs are presented in Section 2. The proposed psoCNN algorithm is described in details in Section 3. The experimental design, such as datasets, peer competitors models and algorithm parameters, is presented in Section 4. The experimental results and a comparison with chosen algorithms are presented in Section 5. For last, Section 6 concludes this paper and proposes directions for future works.

2. Background and motivation

2.1. Particle swarm optimization

Particle Swarm Optimization (PSO) is a meta-heuristic algorithm often used in discrete, continuous and combinatorial optimization problems. It was first developed by Kennedy and Eberhart in 2001 [30]. It is inspired by the flying pattern of a flock of birds. In the context of PSO, a single solution is called a particle, and the collection of all solutions is called a swarm. The main idea in PSO is that each particle only has knowledge about its current velocity, its own best configuration achieved in the past (*pBest*), and which particle is the current global best in the swarm (*gBest*). At every iteration, each particle adjusts its velocity in a way that its new position will be closer to *gBest* and its *pBest* at the same time. The velocity of each particle, v, is updated according to the following equation:

$$v_{i,j}(t+1) = w \times v_{i,j}(t) + c_p \times r_p \times (pBest_{i,j} - x_{i,j}(t)) + c_g \times r_g \times (gBest_j - x_{i,j}(t)), \quad (1)$$

where $v_{i,j}$ is the velocity of the i-th particle in the j-th dimension, x is the current particle position, w is a constant called momentum that adjusts how much the velocity from the previous time step will affect the velocity at the current time step, c_p, and c_g are constants defined beforehand, and r_p and r_g are random numbers in $[0, 1)$. Moreover, the

algorithm ability for exploration and exploitation can be tuned by modifying the constants c_p and c_g, respectively.

Finally, the position of the i-th particle in the j-th dimension is updated as follows:

$$x_{i,j}(t+1) = x_{i,j}(t) + v_{i,j}(t+1). \tag{2}$$

One of the most positive characteristics of PSO is that it converges faster than GAs [31,32]. Due to the fact that a single training run of a CNN can take up a couple of days even in the most powerful computers, this characteristic can have a positive impact when searching for optimal CNN architectures.

2.2. Deep convolutional neural networks

Deep neural networks (DNNs) can be categorized in feed-forward neural networks, such as fully-connected neural networks (FCNNs) and deep convolutional neural networks (CNNs) [33], and recurrent neural networks, such as long short-term memory (LSTMs) [34] and gated recurrent units (GRUs) [35]. As the name suggests, feed-forward neural networks mainly process information layer by layer, mostly suitable in classification and object detection tasks, while recurrent neural networks have feedback loops between layers allowing them to be used in time-dependent tasks, such as natural language understanding.

In CNNs, layers are stacked together such that the output of any given layer will be the input to the next layer [33,36]. The output of each layer is a function of its inputs and internal weights. Mathematically, a CNN is defined as in Eq. (3), where $\mathbf{X}$ is the input data, $f_i(\cdot)$ is the activation function of the i-th layer, $g_i(\cdot)$ is the weight operation of the i-th layer, $\mathbf{Z}_i$ is the output of the weight operation of the i-th layer before the activation function, $\mathbf{W}_i$ denotes the weight of the i-th layer, and $\mathbf{O}_i$ is the output of the i-th layer. There are mainly three types of layers: convolution (*conv*), pooling (*pool*) and fully-connected (*FC*) layers, as shown in Eq. (4). The output of a convolution layer is a convolution operation ($\circledast$) on its inputs and weights, also known as kernels or filters. A pooling layer can be max pooling or average pooling, and it performs a down-sampling operation in a $r \times c$ window ($\boxplus_{r,c}$) of its input reducing the number of its output parameters. A fully-connected layer works like traditional ANNs, where its output is a function of the weights multiplied by its input.

$$\begin{cases} \mathbf{O}_i = \mathbf{X} & i = 1 \\ \mathbf{O}_i = f_i(\mathbf{Z}_i) & i > 1 \\ \mathbf{Z}_i = g_i(\mathbf{O}_{i-1}, \mathbf{W}_i) & \end{cases} \tag{3}$$

$$\begin{cases} \mathbf{Z}_i = \mathbf{W}_i \circledast \mathbf{O}_{i-1} & \text{if } i-\text{th layer is } conv \\ \mathbf{Z}_i = \boxplus_{r,c}\mathbf{O}_{i-1} & \text{if } i-\text{th layer is } pool \\ \mathbf{Z}_i = \mathbf{W}_i\mathbf{O}_{i-1} & \text{if } i-\text{th layer is } FC \end{cases} \tag{4}$$

When training neural networks, the main objective is to minimize the error between training targets and the network predicted outputs. In the case of CNNs, it is common to minimize the cross-entropy loss. In general, the minimization is carried out by using gradient descent and backpropagation [33,36].

Due to the fact that even simple CNNs can have millions of parameters, training is only feasible when using graphic processing units (GPUs). Depending on the CNN architecture, one training run can take up days or even weeks, making it very time consuming to trial and error multiples CNN architectures. Thus, it is important to develop algorithms capable of automatically creating and evaluating CNN architectures as fast as possible.

3. Proposed algorithm

The proposed algorithm receives as inputs parameters related to the problem in question, such as the training data that will be used and parameters related to the CNN architectures that will be created, such as the particles' maximum number of layers during initialization. The general framework of the proposed psoCNN can be seen in Algorithm 1. In the proposed algorithm, the global best particle is based on the best blocks found in the swarm by following the PSO algorithm. Thus, there is no need to manually optimize the parameters of each block. In our proposed approach, the parameter optimization does not start over. Good blocks are passed along to the next generation in the form of the global best particle. Currently, only the particle evaluation that needs to be restarted at every iteration, but the algorithm ensures that good blocks are kept.

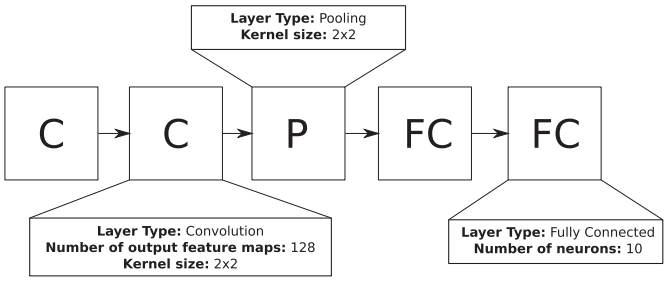

Fig. 1. Representation of a single particle in the proposed psoCNN.

The proposed algorithm consists of a PSO framework, including six procedures that allow it to be used to search for optimal CNN architectures: an efficient CNN representation, initialization of a swarm of particles, fitness evaluation of individual particles, a measure of the difference between two particles, velocity computation and particle update. These operations are detailed in the following subsections.

3.1. Convolutional neural networks representation

Representation is the most important aspect when designing an algorithm that deals with complex structures, such as CNN architectures. We propose a direct encoding scheme for CNN architectures. Every time a CNN is used for training, testing or evaluation, a model is compiled following an array containing the details of each layer in the particle. The proposed algorithm only searches for CNN architectures. The weights of each layer are not considered searchable or evolvable in the current study.

There are four types of layers: convolutional, max pooling, average pooling, and fully-connected. Each position in the list of layers contains information regarding its type and hyperparameters. For example, a single position in this list will contain the layer type, the number of output feature maps, if convolutional, or the number of neurons, if fully-connected, and the kernel size, if convolutional or pooling. Fig. 1 illustrates the representation used in the psoCNN algorithm, where C, P, and FC stand for convolutional, pooling (max or average) and fully-connected layers, respectively. One important characteristic of the algorithm is that the particles are used in the PSO without the need to convert their information to numerical values. Furthermore, in the proposed representation, each particle can be thought as a series of functional blocks which could allow the use of custom blocks. Currently, the blocks used in the proposed psoCNN are the four layers types showed previously.

3.2. Initialization of the swarm

The initialization of the swarm (*InitializeSwarm*()) is the first step in the proposed psoCNN. This function will create N particles with random CNN architectures, and it is detailed in Algorithm 2. Each particle will have a random number of layers, between three up to $l_{\max}$ (the maximum number of layers). In order to generate *feasible* CNN architectures, the first and last layer of every particle are always a convolution and a fully-connected layer, respectively. In addition, fully-connected layers (FC) cannot be placed between convolutions or pooling layers, but only

at the end of the architecture. Thus, during initialization, the algorithm needs to make sure that once a FC layer is added to the architecture, every layer following it is also a FC layer. The use of FC layers at the end of a convolutional neural network (CNN) is a convention adopted by other researchers in the field. Typically, FC layers are used to classify the features extracted by the convolutional and pooling layers. Using fully-connected layers between convolutional and pooling layers would increase the number of parameters of the overall CNN. This comes from the fact that after each pooling layer the number of outputs is reduced by a factor of two. After a certain number of pooling layers, the outputs will be small enough to be used as input to the FC layers without the need of using lots of neurons.

In Algorithm 2, the *addConv*() function will add a convolutional layer to the particle architecture with a random number of output feature maps from 1 up to $maps_{\max}$, its kernel size will be chosen at random from 3×3 up to $k_{\max} \times k_{\max}$, where $k_{\max}$ denotes the maximum convolution kernel size, and its kernel strides will always be 1×1. The *addPool*() function will add at random a maximum pooling or an average pooling layer to the particle architecture with a window size of 3×3, and a stride of 2×2. The *addFC*() function will add a fully-connected layer to the particle architecture with a number of hidden neurons between 1 up to a maximum of $n_{\max}$. The activation function of all layers is always a rectified linear unit (ReLU). Finally, the number of pooling layers is constrained by the size of the input data. For example, for an input of size 28×28, the algorithm can only add up to three pooling layers.

3.3. Fitness evaluation

The fitness evaluation is performed by the function *ComputeLoss*(). It is done by compiling the particle architecture into a full-fledged CNN and training it for a total of e_{train} epochs. The evaluation itself is done by comparing the loss function of each particle, in our case the cross-entropy loss. Thus, the objective of the algorithm is to find a particle architecture with the smallest loss, regardless of the number of parameters or other criteria. Training is performed using Adam [37] and weights are initialized with Xavier [38]. Furthermore, it is also possible to add dropout and batch normalization between layers avoiding the overfitting problem [39]. This is the main bottleneck of the proposed algorithm because every one of the N particles needs to be trained in the entire dataset.

3.4. Measuring difference between particles

Before being able to compute the velocity of a single particle, we need to specify how to measure the difference between two particles. An overview of this procedure can be seen in Fig. 2, where two particles, $P1$ and $P2$, shown in the top-left corner, are compared. In order to avoid ending up with fully-connected layers between convolutions and pooling (Conv/Pool) layers, and, therefore, with an invalid CNN architecture, the Conv/Pool and FC layers of each particle are compared independently, as shown in the top-right corner of Fig. 2. The difference only takes into consideration the type of layer of each particle. For example, if the second layer of both particles is a convolution layer, the difference will be zero, independent of the hyperparameters, which indicates to the velocity operator that this layer position should not change when updating a given particle architecture. In addition, the comparison is always with respect to the first particle. If both particles have different layers type, the result will be to keep the layer from the first particle with its corresponding hyperparameters. If the first particle has fewer layers than the second one, a -1 will be added to the final difference, meaning that this position should be removed. On the other hand, if the first particle has more layers than the second one, a $+L$ will be added to the final difference, where L indicates the layer type, C, P, or FC, to be added.

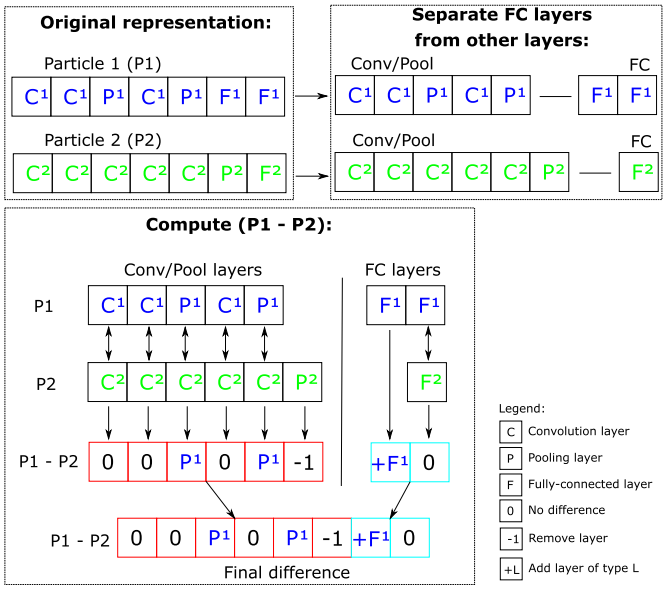

Fig. 2. Example of the difference between two particles.

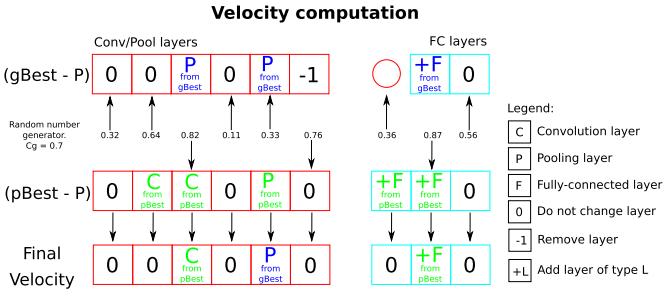

Fig. 3. Example of the velocity computation of a single particle.

3.5. Velocity computation

The velocity of any given particle (P) is based on the difference with respect to the global best (*gBest*), and the particle best (*pBest*). Thus, we need to compute two differences: ($gBest - P$) and ($pBest - P$). In Algorithm 1, this procedure is done in the function *UpdateVelocity*(). An example of such computation is shown in Fig. 3, where the two first rows represent the differences ($gBest - P$) and ($pBest - P$), with the difference from Conv/Pool and FC layers showed separately. The final velocity is computed by drawing a number r uniformly at random from $[0, 1)$, and by choosing a layer from one of the two differences based on the decision factor C_g. If $r \leq C_g$, the algorithm will select a layer from the difference ($gBest - P$). Otherwise, the algorithm will select a layer from the difference ($pBest - P$). Thus, the decision factor C_g will control how fast the updated particle architecture will approach the *gBest* architecture. Similar to the difference computation, the velocity computation is done by separating Conv/Pool layers from FC layers.

However, there is one special case that can occur when computing a particle's velocity. Since the velocity is computed by comparing only the layers types, it is possible that a particle has the exactly same layers types as the swarm's *gBest* and its *pBest*. In this case, the proposed difference procedure will return a list of 0's, as shown in the top part of Fig. 4. When this happens, the velocity will be computed by selecting layers from *gBest* or *pBest* according to the decision factor C_g. Therefore, the particle's hyperparameters will be chosen from *gBest* or *pBest* instead of the difference, as shown in Fig. 4.

Algorithm 1 Proposed psoCNN.

Input : Swarm size (N), number of iterations ($iter_{max}$), training data (X), maximum number of layers (l_{max}), probability of choosing $gBest$ over $pBest$ when computing velocities (Cg), maximum convolution kernel size (k_{max}), maximum number of neurons in a FC layer (n_{max}), number of outputs (n_{out}), number of training epochs during particle evaluation (e_{train}), number of training epochs for the best CNN found (e_{test}).

Output: The best CNN architecture found.

$S = \{P_1, ..., P_N\} \leftarrow InitializeSwarm(N, l_{max}, k_{max}, n_{max}, n_{out})$;
$P_1.pBest \leftarrow P_1$;
$P_1.loss, P_1.pBest.loss \leftarrow ComputeLoss(P_1, X, e_{train})$;
Initialize $gBest \leftarrow P_1$;
$gBest.loss \leftarrow P_1.loss$;
for $i = 2$ *to* N **do**
  $P_i.pBest \leftarrow P_i$;
  $P_i.loss, P_i.pBest.loss \leftarrow ComputeLoss(P_i, X, e_{train})$;
  if $P_i.loss \leq gBest.loss$ **then**
    $gBest \leftarrow P_i$;
  end
end
for $iter = 1$ *to* $iter_{max}$ **do**
  for $i = 1$ *to* N **do**
    $P_i.velocity \leftarrow UpdateVelocity(P_i, Cg)$;
    $P_i \leftarrow UpdateParticle(P_i)$;
    $P_i.loss \leftarrow ComputeLoss(P_i, X, e_{train})$;
    if $P_i.loss \leq P_i.pBest.loss$ **then**
      $P_i.pBest \leftarrow P_i$;
      $P_i.pBest.loss \leftarrow P_i.loss$;
      if $P_i.pBest.loss \leq gBest.loss$ **then**
        $gBest \leftarrow P_i$;
        $gBest.loss \leftarrow P_i.loss$;
      end
    end
  end
end
$gBest \leftarrow ComputeLoss(gBest, X, e_{test})$;
return $gBest, gBest.loss$;

3.6. Particle update

Updating the architecture of a particle is the most straightforward procedure in the proposed psoCNN algorithm. The update occurs in *UpdateParticle()* in Algorithm 1. Once again, Conv/Pool layers are treated independent of FC layers. The algorithm follows the given particle's *velocity* searching for layers that should be modified. Layers are added or removed from the particle architecture according to its *velocity*. However, the algorithm needs to keep track of the number of pooling layers in the architecture. Depending on the size of the training inputs, only a finite number of pooling layers is allowed. If, after the update, the particle ends up with more pooling layers than that allowed, pooling layers in excess are removed from the particle's architecture from the last to the first layer one by one. This process is illustrated in Fig. 5.

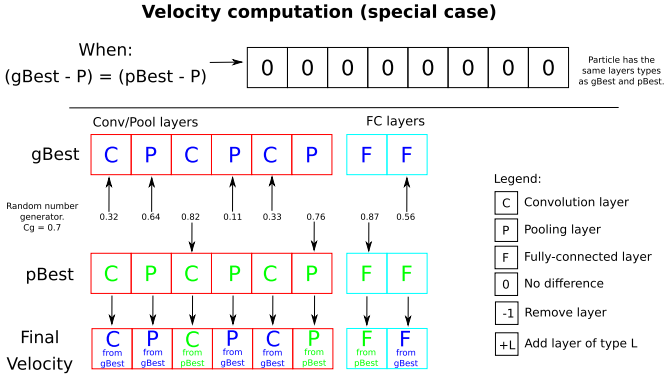

Fig. 4. Example of the velocity computation when the particles has the same layers types as *gBest* and *pBest*.

4. Experimental design

The proposed algorithm, psoCNN, is tested with state-of-the-art image classification datasets. The following subsections present in details the datasets, peer competitors models and the algorithm parameters used in our experiments. In addition, we are making the code of the proposed algorithm publicly available to other researchers by request allowing the reproduction of the results presented in this paper.

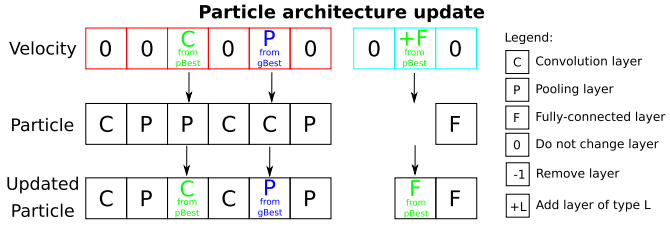

Fig. 5. Example of a particle architecture update.

Algorithm 2 Initialization of the swarm in the proposed psoCNN (*InitializeSwarm*()).

Input : Swarm size (N), maximum number of layers (l_{max}), maximum number of feature maps ($maps_{max}$), maximum convolution kernel size (k_{max}), maximum number of neurons in a FC layer (n_{max}), number of outputs (n_{out}).

Output: A set of N particles $S = \{P_1, ..., P_N\}$.

for $i = 1$ *to* N **do**
  $P_i.depth = rand(3, depth)$;
  for $j = 1$ *to* $P_i.depth$ **do**
    if $j == 1$ **then**
      $list\_layers[j] \leftarrow addConv(k_{max}, maps_{max})$;
    else if $j == P_i.depth$ **then**
      $list\_layers[j] \leftarrow addFC(n_{out})$;
    else if $list\_layers[j-1].type == \text{“}fully - connected\text{”}$ **then**
      $list\_layers[j] \leftarrow addFC(n_{max})$;
    else
      $layer\_type \leftarrow rand(1, 3)$;
      if $layer\_type == 1$ **then**
        $list\_layers[j] \leftarrow addConv(k_{max}, maps_{max})$;
      else if $layer\_type == 2$ **then**
        $list\_layers[j] \leftarrow addPool()$;
      else
        $list\_layers[j] \leftarrow addFC(n_{max})$;
      end
    end
  end
  $P_i.list\_layers \leftarrow list\_layers$;
end
return $S = \{P_1, ..., P_N\}$;

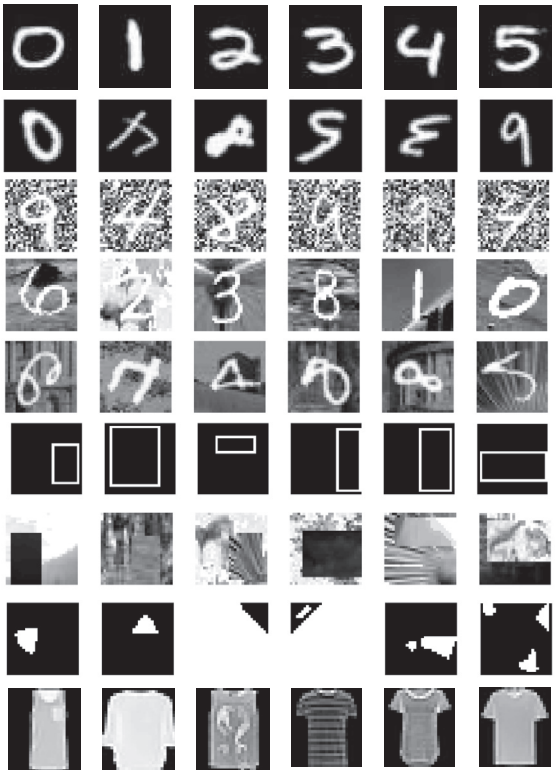

Fig. 6. Sample pictures of each dataset. From top to bottom: MNIST, MNIST-RD, MNIST-RB, MNIST-BI, MNIST-RD + BI, Rectangles, Rectangles-I, Convex, and MNIST-Fashion.

Table 1
Overview of the datasets used for evaluating the psoCNN.

Dataset	Input size	# classes	# training	# test
MNIST	28 × 28 × 1	10	60,000	10,000
MNIST-RD	28 × 28 × 1	10	12,000	50,000
MNIST-RB	28 × 28 × 1	10	12,000	50,000
MNIST-BI	28 × 28 × 1	10	12,000	50,000
MNIST-RD + BI	28 × 28 × 1	10	12,000	50,000
Rectangles	28 × 28 × 1	2	1,200	50,000
Rectangle-I	28 × 28 × 1	2	12,000	50,000
Convex	28 × 28 × 1	2	8,000	50,000
MNIST-Fashion	28 × 28 × 1	10	60,000	10,000

4.1. Datasets

Nine datasets with publicly available results are chosen to evaluate the proposed algorithm and compare it with others deep learning models. They are the following: *MNIST*, *MNIST-RD*, *MNIST-RB*, *MNIST-BI*, *MNIST-RD + BI*, *Rectangles*, *Rectangles-I*, *Convex*, and *MNIST-Fashion*. Some sample pictures of each dataset can be seen in Fig. 6, and an overview of their input, training, and test sizes is shown in Table 1. Due to our limited amount of computing power and our focus on developing efficient evolutionary algorithm scalable for large datasets in the near future, the algorithm was tested only with datasets with small input size.

The first one is the *MNIST* (Modified National Institute of Standards and Technology) dataset created by LeCun et al. in 1989 [40]. It is a well-known dataset used to test new machine learning algorithms. It contains black and white images of handwritten digits from zero to nine with a total of 50,000 training samples and 10,000 test samples.

The *MNIST-RD* (rotated digits), *MNIST-RB* (random noise as background), *MNIST-BI* (background images), *MNIST-RD + BI* (rotated digits and background images) are modified versions of the original *MNIST* dataset created by Larochelle et al. [41]. These datasets are considered more challenging than the original *MNIST* because they test how well a model can generalize from data containing irrelevant information to the problem. They all contain 12,000 training samples and 50,000 test samples. The reduced number of training samples also challenges the models to learn only the meaningful features from data.

The *Rectangles* dataset contains black and white images of rectangles with different widths and heights. The images contain only the rectangle outline with the inside having the same color as the background. In this dataset, the objective of the model is to learn which dimension is larger: the width or the height. It contains 1,200 training samples and 50,000 test samples. The *Rectangles-I* (Rectangles with Images) dataset has one key difference: either the rectangle or the background contains image patches. The added images make the classification task much difficult for any algorithms. The *Rectangles-I* contains 12,000 training images and 50,000 test images. The *Convex* dataset contains black and white images of geometrical shapes where the model has to determine if the shape is convex or not. It contains 8,000 training samples and 50,000 test samples.

The *MNIST-Fashion* [42] contains 60,000 training samples and 10,000 test samples. Its inputs are back and white pictures of fashion garments with size of 28 × 28, similar to the original *MNIST* input size. It also contains 10 classes: *t-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag,* and *ankle boot*.

4.2. Peer competitors' models

In order to evaluate the performance of the proposed psoCNN algorithm, we choose to compare our results with those of state-of-the-art deep learning algorithms from our peers competitors in this field. We choose competing algorithms that have reported their results in the same datasets we used to test the proposed psoCNN. It is important to notice that not all competitors are population-based algorithms, and it is not possible to make a direct comparison between the computational complexity of the proposed psoCNN algorithm and many peer competing algorithms.

The only population-based algorithms we used to compare our results are the evoCNN [19] and IPPSO [29] algorithms. These algorithms are the most similar to our proposed psoCNN where a population of individuals is evolved over time to find good CNN models for a given dataset. They are the only algorithms where we can make a direct comparison of computational complexity with the proposed one. All the other algorithms showed in our results are deep learning models hand-crafted for the problem in question. It is not possible to compare the computational complexity of our algorithm with hand-crafted ones. For all other models presented, we compare how well the CNNs found by psoCNN performed in the chosen test sets when compared with the manually designed ones.

The *MNIST* dataset is the most common one used to test deep learning algorithms, and many of our peers reported their results in this dataset. We choose to compare our results with the ones obtained by the first convolutional neural networks created by LeCunn et al. in 1998, known as *LeNet-1*, *LeNet-4*, and *LeNet-5* [33]. These are simple CNNs designed to tackle the problem of digit recognition, and have been surpassed by modern CNNs. We also compare our results with modern CNNs, such as RCNN (Recurrent Convolutional Neural Network) [43] and DropConnect [44]. Liang and Hu created the RCNN by adding recurrent connections in each layer of a CNN which obtained state-of-the-art results in many datasets. DropConnect was developed by Wan et al., and it is a generalization of traditional Dropout. In traditional Dropout, activations are chosen at random to be dropped in fully-connected layers. In DropConnect, the weights are dropped at random in fully-connected layers.

For the *MNIST-RD*, *MNIST-RB*, *MNIST-BI*, *MNIST-RD + BI*, *Rectangles*, *Rectangles-I*, and *Convex* datasets, we choose to compare our results with *CAE-1*, *CAE-2*, *PCANet-2*, *RandNet-2*, *LDANet-2*, *evoCNN*, and *IPPSO*. Note that we are only reporting the results disclosed by the authors of each algorithm. Developed by Rifay et al. CAE-1 and CAE-2 stand for Contractive Auto-Encoder with one and two layers [45], respectively. CAE is an Auto-Encoder trained with a penalty term that results in a localized space contraction with state-of-the-art results when an ANN is initialized with such technique. PCANet-2, RandNet-2 and LDANet-2 were created by Chan et al. [46], and it is a modified DNN where features are extracted using Principal Component Analysis (PCANet-2), random cascaded filters (RandNet-2), or Linear Discriminant Analysis (LDANet-2). For the *MNIST-Fashion* dataset, we choose more traditional convolutional neural networks and the evoCNN algorithms to compare our results with.

4.3. Algorithm parameters

The parameters used in the proposed psoCNN can be grouped in three categories: *particle swarm optimization*, *CNN architecture initialization*, and *CNN training*. The parameters used for all datasets can be found in Table 2, and their effect in the proposed algorithm is discussed in the following paragraphs.

Table 2
Algorithm parameters used to evaluate the psoCNN.

Description	Value
Particle swarm optimization	
Number of iterations	10
Swarm size	20
Cg	0.5
CNN architecture initialization	
Minimum number of outputs from a Conv layer	3
Maximum number of outputs from a Conv layer	256
Minimum number of neurons in a FC layer	1
Maximum number of neurons in a FC layer	300
Minimum size of a Conv kernel	3 × 3
Maximum size of a Conv kernel	7 × 7
Minimum number of layers	3
Maximum number of layers	20
CNN training	
# epochs for particle evaluation	1
# epochs for the global best	100
Dropout rate	0.5
Batch normalize layer outputs	Yes

The parameters used in the first category control the behavior of the PSO algorithm. It contains three parameters: the number of iterations, the swarm size, and the probability of choosing a layer from the global best when computing a particle's velocity (C_g). The *number of iterations* controls the total number of iterations that the proposed algorithm will run before considering that the optimization is finished. The best CNN architecture found by the algorithm is saved in the local disk at the end of the optimization. The *swarm size* defines how many particles will be used by the PSO algorithm. In our case, each particle is one CNN architecture to be tested by the algorithm. To better explore the space of possible architectures, it is preferable to use as many particles as possible during the search process. The C_g parameters will effectively control how fast the particles in the swarm will approach the global best particle. In our case, a higher C_g will make the particles' architecture resemble the global best as quickly as possible. In the proposed algorithm, higher C_g values will reduce the diversity of the swarm and, consequently, increase the probability of getting trapped in a local optima architecture.

The parameters used in the second category control the initial diversity of our particles' architectures. It contains a total of eight parameters: the minimum number of outputs from a convolutional layer, the maximum number of outputs from a convolutional layer, the minimum number of neurons in a fully-connected layer, the maximum number of neurons in a fully-connected layer, the minimum size of a convolutional kernel, the maximum size of a convolutional kernel, the minimum number of layers, and the maximum number of layers. The first step in the proposed algorithm is to create an initial population, or swarm, of CNN architectures. This initial population will contain individuals with architectures chosen at random bounded by these parameters. The *minimum and maximum number of outputs from a convolutional layer* will limit the number of feature maps that any given convolutional layer can output. Likewise, *the minimum and maximum number of neurons in a fully-connected layer* will constraint the number of neurons that any given FC layer can have. The size of the convolutional kernel will always be a number between the *minimum and maximum size of a convolutional kernel*. Finally, the depth of a particle's architecture will be chosen at random and bounded by the *minimum and maximum number of layers* parameters. It is important to notice that these parameters only control the initial architecture of any given particle. Once initialized, the particle's architecture will be updated following our proposed psoCNN algorithm. For example, a particle can have its number of layers reduced or increased during the optimization process. However, these parameters

Table 3
psoCNN test errors for the MNIST, MNIST-RD, MNIST-RB, MNIST-BI, MNIST-RD + BI, Rectangles, Rectangles-I, and Convex datasets compared to different models.

Model	MNIST	MNIST-RD	MNIST-RB	MNIST-BI	MNIST-RD + BI	Rectangles	Rectangles-I	Convex
LeNet-1 [33]	1.7% (+)	–	–	–	–	–	–	–
LeNet-4 [33]	1.1% (+)	–	–	–	–	–	–	–
LeNet-5 [33]	0.95% (+)	–	–	–	–	–	–	–
RCNN [43]	0.31% (−)	–	–	–	–	–	–	–
DropConnect [44]	0.21% (−)	–	–	–	–	–	–	–
CAE-1 [45]	2.83% (+)	11.59% (+)	13.57% (+)	16.7% (+)	48.10% (+)	1.48% (+)	21.86% (+)	–
CAE-2 [45]	2.48% (+)	9.66% (+)	10.90% (+)	15.5% (+)	45.23% (+)	1.21% (+)	21.54% (+)	–
PCANet-2 [46]	1.06%(+)	8.52% (+)	6.85% (+)	11.55% (+)	35.86% (+)	0.49% (+)	13.39% (+)	4.19% (+)
RandNet-2 [46]	1.27% (+)	8.47% (+)	13.47% (+)	11.65% (+)	43.69% (+)	0.09% (+)	17.00% (+)	5.45% (+)
LDANet-2 [46]	1.40% (+)	4.52% (+)	6.81% (+)	12.42% (+)	38.54% (+)	0.14% (+)	16.20% (+)	7.22% (+)
evoCNN (best) [20]	1.18% (+)	5.22% (+)	2.8% (+)	4.53% (+)	35.03% (+)	0.01% (−)	5.03% (+)	4.82% (+)
evoCNN (mean) [20]	1.28% (+)	5.46% (+)	3.59% (+)	4.62% (+)	37.38% (+)	0.01% (−)	5.97% (+)	5.39% (+)
IPPSO (best) [29]	1.13% (+)	–	–	–	34.50% (+)	–	–	8.48% (+)
IPPSO (mean) [29]	1.21% (+)	–	–	–	33% (+)	–	–	12.06% (+)
psoCNN (best)	**0.32%**	**3.58%**	**1.79%**	**1.90%**	**14.28%**	**0.03%**	**2.22%**	**1.7%**
psoCNN (mean)	**0.44%**	**6.42%**	**2.53%**	**2.40%**	**20.98%**	**0.34%**	**3.94%**	**3.9%**

will also control the size of the searching space for optimization. If one wants to search for very large and deep CNN architectures, he or she must increase the values of all parameters presented here. However, the use of large and deep architectures will be limited by the amount of video memory available in the system running the proposed algorithm. The psoCNN was tested on a single Nvidia Tesla P100 GPU with 16 GB of video memory. Despite being one of the most powerful GPUs for training deep neural networks, the amount of video memory is far from enough to train CNNs with more than 5 million parameters.

For last, the parameters used in the third category control the training process of each particle. It contains a total of four parameters: the number of epochs for particle evaluation, the number of epochs for the global best, the dropout rate, and the choice of using batch normalization (BN) between layers. These parameters will only control

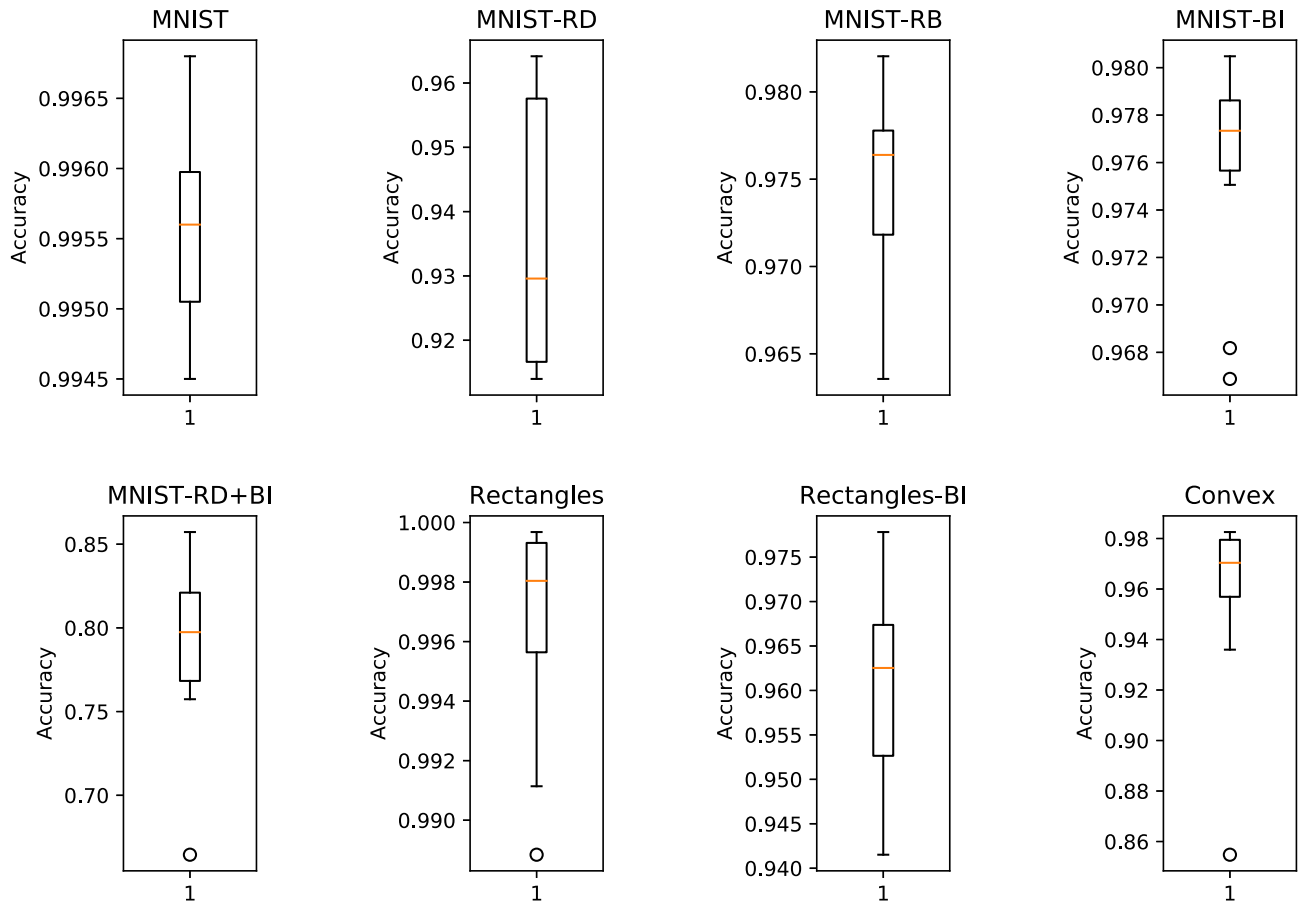

Fig. 7. Boxplots of the test accuracy for the MNIST, MNIST-RD, MNIST-RB, MNIST-BI, MNIST-RD + BI, Rectangles, Rectangles-I and Convex datasets obtained with the proposed psoCNN.

Table 4
psoCNN test errors for the MNIST-Fashion dataset compared to different models.

Model	Test error	# parameters
Human performance[a]	16.5% (+)	–
MLP 256-128-100[a]	11.67% (+)	3 M
GRU + SVM[a]	11.2% (+)	–
HOG + SVM[a]	7.4% (+)	–
AlexNet[a]	10.1% (+)	62.3 M
3CONV + 3FC[a]	6.6% (+)	500 k
VGG16[a]	6.5% (+)	26 M
GoogLeNet[a]	6.3% (+)	23 M
evoCNN (best) [20]	5.47% (−)	6.68 M
evoCNN (mean) [20]	7.28% (+)	6.52 M
MobileNet[a]	5% (−)	4 M
psoCNN − dropout − BN (best)	8.1% (+)	1.4 M
psoCNN − dropout − BN (mean)	9.15% (+)	1.8 M
psoCNN + dropout + BN (best)	**5.53%**	**2.32M**
psoCNN dropout + BN (mean)	**5.90%**	**2.5M**

[a] https://github.com/zalandoresearch/fashion-mnist.

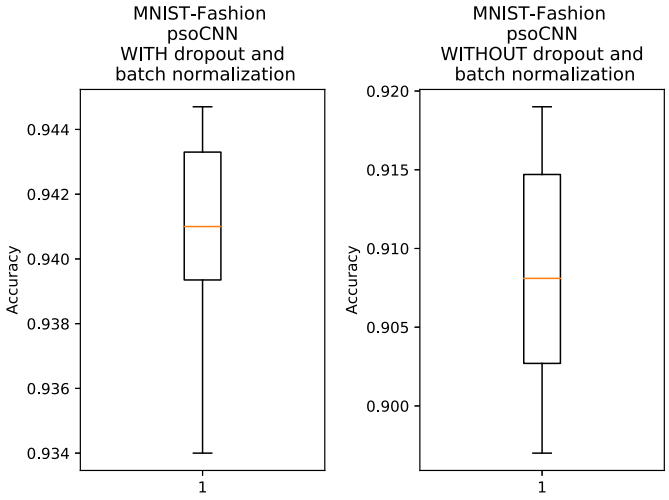

Fig. 8. Boxplots of the test accuracies for the MNIST-Fashion dataset obtained with the proposed psoCNN.

the weight updating process during particle evaluation. The *number of epochs for particle evaluation* will control how many times the particle will be trained in the complete dataset before its training accuracy is used for evaluation. At the end of the optimization, the global best particle found by the algorithm will be trained for *number of epochs for the global best* before such particle is evaluated in the test set. The *dropout rate* parameter is used during any particle training to avoid overfitting by dropping a percentage of the connections between neurons in fully-connected layers at random. Finally, the proposed algorithm allows the use of batch normalization between the layers of a particle in order to further avoid overfitting during the training process. However, due to time constraint, we only train and evaluate the *MNIST-Fashion* dataset with and without dropout and BN. All other datasets are trained and evaluated with a dropout rate of 0.5 and BN. For last, in order to obtain statistically meaningful results, all tests consist of a total of 30 independent runs.

5. Experimental results and analysis

This section presents the results obtained with the psoCNN algorithm compared with other deep learning algorithms, and a discussion of the results. Our reported results for each of the datasets were obtained using only the test set. This ensures that the algorithm can learn and generalize well.

5.1. Experimental results

For the *MNIST*, *MNIST-RD*, *MNIST-RB*, *MNIST-BI*, *MNIST-RD + BI*, *Rectangles*, *Rectangles-I* and *Convex* datasets, the best test errors obtained by the proposed psoCNN, shown in Table 3, are 0.32%, 3.58%, 1.79%, 1.90%, 14.28%, 0.03%, 2.22% and 1.7%, respectively. While the mean test errors on these datasets are 0.44%, 6.42%, 2.53%, 2.40%, 20.98%, 0.34%, 3.94%, and 3.9%, respectively. A boxplot showing the test accuracy distribution on these datasets can be found in Fig. 7.

Although our results are not the best on the *MNIST* dataset with respect to chosen competitors, they are within the top eight best results reported by Rodrigo Benenson [47]. Considering that we are only using batch normalization and dropout without any kind of data augmentation, skip connection or parallel layers, our results are considered one of the best on *MNIST* using simple convolutional neural net-

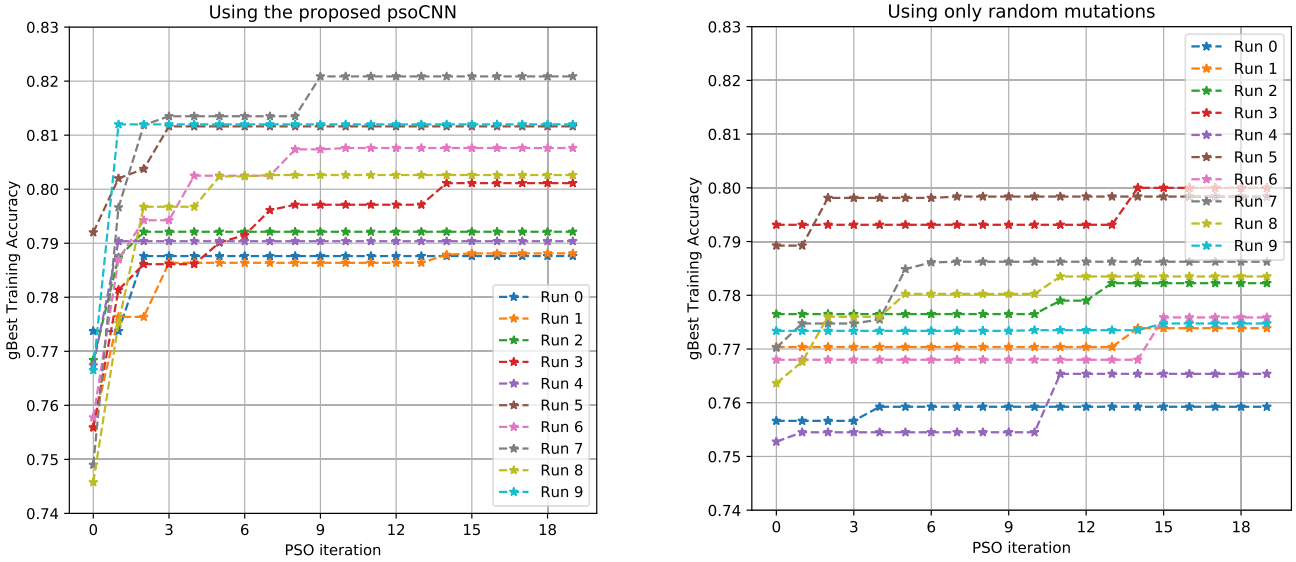

Fig. 9. Evolution of the global best particle training accuracy on the Convex dataset for 10 runs. Left: using the proposed psoCNN. Right: using only random mutations.

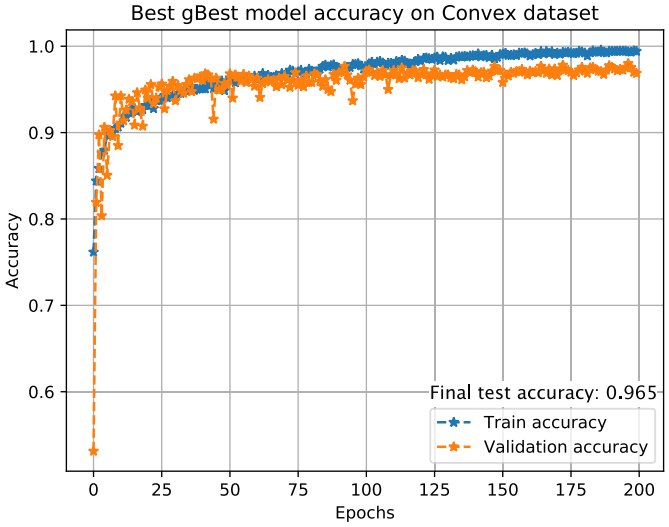

Fig. 10. Training curves for the best gBest model found by psoCNN on the Convex dataset.

works architectures. For example, Liang and Hu [43] were able to achieve a test error of 0.31% using recurrent convolutional neural networks (RCNN). Currently, the best test error obtained is 0.21% using neural networks with a regularization technique called DropConnect [44].

Currently, the best results for *MNIST-RD*, *MNIST-RB*, *MNIST-BI*, *MNIST-RD + BI*, *Rectangles*, *Rectangles-I*, and *Convex* are from the evoCNN algorithm developed by Yanan et al. [19]. However, from Table 3, we can see that the proposed algorithm outperforms evoCNN on all, but the *Rectangles* dataset.

The results for the *MNIST-Fashion* dataset can be found in Table 4 and Fig. 8. With this dataset, we test the algorithm with and without dropout and batch normalization which are indicated by "*psoCNN + dropout + BN*" and "*psoCNN − dropout − BN*" in Table 4, respectively. The best test error obtained is 5.53% when using dropout and batch normalization. The mean test error is 5.90% when using dropout and batch normalization, and 9.15% without using dropout and batch normalization. These results show that the proposed psoCNN is able to achieve one of the top results on this dataset. The proposed psoCNN outperforms more complex networks such as AlexNet [48], VGG16 [3], and GoogLeNet [4].

5.2. Results discussion

The models found by psoCNN outperform state-of-the-art models in six out of the nine datasets. psoCNN is able to find models with significantly less parameters than those of other models. For example, in the *MNIST-Fashion* dataset, the best model found only has 2.32 million parameters (see Table 4) which is much less than those of evoCNN, VGG16, GoogleNet, and AlexNet models. Furthermore, the algorithm is able to achieve competitive results without using any data augmentation techniques, and without using any complex architectures. Currently, the models created by the proposed psoCNN do not feature feedback or parallel connections and are a lot simpler than those of other models. The proposed algorithm is able to find smaller networks than peer competitors' models because the swarm was initialized with small networks. This highlights the fact that many hand-crafted networks contain too many unnecessary parameters, and it is well possible to find state-of-the-art models using smaller networks. Furthermore, smaller networks converge faster than larger ones. Therefore, most population-based methods will not be able to find good larger networks because they are eliminated early in the process before they have any chance of performing better than the smaller ones. One of our future research direction is to create an algorithm where test accuracies and the number of parameters are optimized at the same time.

The proposed algorithm is also consistently improving the training accuracy of the global best particle with each iteration. In the left side of Fig. 9 it is shown the training accuracy for 10 different runs, each one with 20 PSO iterations, on the *Convex* dataset with 30 particles, and all other parameters kept the same as the ones shown in Table 2. We can see that the accuracy is continuously improving at the end of each run. We believe that it would be possible to achieve a better accuracy if we could run the algorithm for more iterations, and with more particles. On the other hand, the right side of Fig. 9 shows the evolution of the global best particle when the difference and update computations are replaced by random mutations where each block has a 20% chance of mutate by deleting a layer, adding a new layer, or changing the block's parameters. We can see that sometimes random mutation can find good architectures, but it takes much longer than the proposed psoCNN. Furthermore, all gBest models found by psoCNN has a training accuracy higher than 0.785, but when using only random mutations only Runs 3 and 5 obtained such training accuracy. This result shows that the proposed algorithm is not performing a simple random searching, and that the difference and update computation designed in this work are indeed working similarly to a PSO algorithm.

In Fig. 10, we show the training curves of the best model found by psoCNN on the *Convex* dataset for 200 epochs. For this test, 20% of the

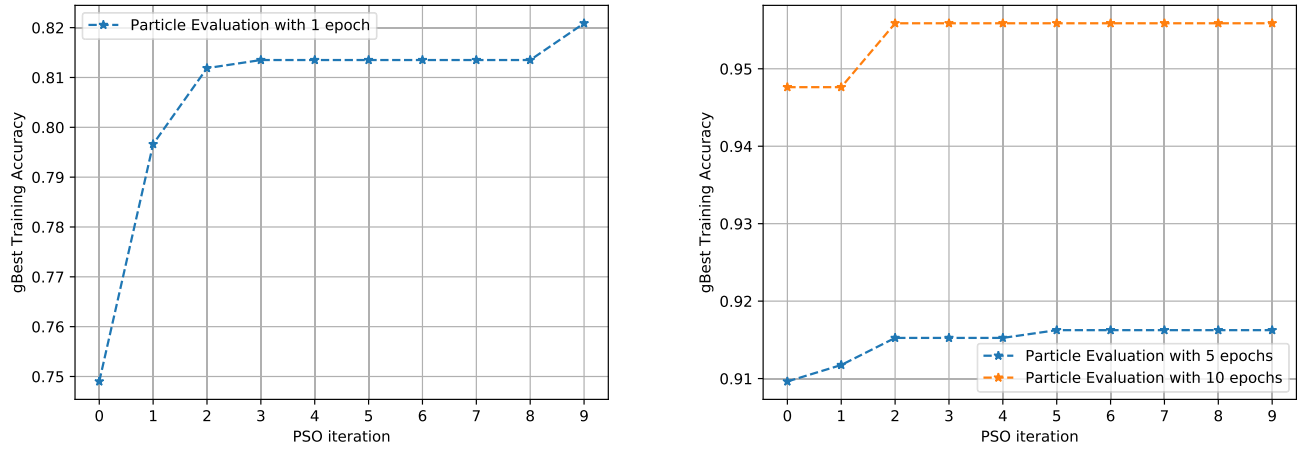

Fig. 11. Effect of the number of epochs used during each particle evaluation on the Convex dataset.

Table 5
Best CNN architectures found by psoCNN on each dataset.

Dataset	Layer	Parameters
MNIST	Convolution	kernel size: 4 × 4; output filters: 77
	Convolution	kernel size: 6 × 6; output filters: 189
	Convolution	kernel size: 5 × 5; output filters: 91
	Convolution	kernel size: 5 × 5; output filters: 185
	Convolution	kernel size: 6 × 6; output filters: 244
	Average Pooling	kernel size: 3 × 3; strides: 2 × 2
	Fully-Connected	output neurons: 10
MNIST-RD	Convolution	kernel size: 6 × 6; output filters: 189
	Convolution	kernel size: 5 × 5; output filters: 182
	Convolution	kernel size: 6 × 6; output filters: 236
	Convolution	kernel size: 5 × 5; output filters: 210
	Average Pooling	kernel size: 3 × 3; strides: 2 × 2
	Fully-Connected	output neurons: 10
MNIST-RB	Convolution	kernel size: 5 × 5; output filters: 246
	Convolution	kernel size: 3 × 3; output filters: 216
	Convolution	kernel size: 5 × 5; output filters: 217
	Convolution	kernel size: 5 × 5; output filters: 156
	Convolution	kernel size: 6 × 6; output filters: 204
	Average Pooling	kernel size: 3 × 3; strides: 2 × 2
	Fully-Connected	output neurons: 10
MNIST-BI	Convolution	kernel size: 3 × 3; output filters: 67
	Convolution	kernel size: 4 × 4; output filters: 126
	Convolution	kernel size: 6 × 6; output filters: 159
	Convolution	kernel size: 3 × 3; output filters: 252
	Convolution	kernel size: 6 × 6; output filters: 202
	Convolution	kernel size: 6 × 6; output filters: 202
	Fully-Connected	output neurons: 10
MNIST-RD + BI	Convolution	kernel size: 4 × 4; output filters: 247
	Convolution	kernel size: 5 × 5; output filters: 208
	Convolution	kernel size: 4 × 4; output filters: 122
	Convolution	kernel size: 5 × 5; output filters: 64
	Convolution	kernel size: 4 × 4; output filters: 144
	Convolution	kernel size: 5 × 5; output filters: 171
	Average Pooling	kernel size: 3 × 3; strides: 2 × 2
	Fully-Connected	output neurons: 10
Rectangles	Convolution	kernel size: 5 × 5; output filters: 139
	Convolution	kernel size: 6 × 6; output filters: 113
	Convolution	kernel size: 5 × 5; output filters: 226
	Fully-Connected	output neurons: 2
Rectangles-I	Convolution	kernel size: 5 × 5; output filters: 127
	Convolution	kernel size: 4 × 4; output filters: 162
	Convolution	kernel size: 6 × 6; output filters: 212
	Convolution	kernel size: 6 × 6; output filters: 212
	Max Pooling	kernel size: 3 × 3; strides: 2 × 2
	Fully-Connected	output neurons: 2
Convex	Convolution	kernel size: 6 × 6; output filters: 118
	Convolution	kernel size: 4 × 4; output filters: 224
	Convolution	kernel size: 5 × 5; output filters: 243
	Convolution	kernel size: 3 × 3; output filters: 71
	Convolution	kernel size: 5 × 5; output filters: 35
	Convolution	kernel size: 5 × 5; output filters: 160
	Fully-Connected	output neurons: 2

training set is used as validation. We can see that training, validation and test accuracies do not show signs of overfitting and they are steadily increasing over time which means that the proposed psoCNN is indeed capable of find good models for a given image classification dataset.

One important parameter is the number of epochs used for each particle evaluation. Fig. 11 shows the effect of using 1, 5 and 10 epochs during particle evaluation. The main issue of using a high number of epochs during particle evaluation is that the architecture searching will take longer to conclude as each particle is a CNN that needs to be trained for this amount of epochs before its quality can be evaluated. If this number is chosen too high, there is the risk of overfitting the data, and the use of the validation error would be recommended instead of the training error.

The proposed algorithm could achieve even better results than the ones presented here. For example, from the parameters showed in Table 2, the models produced by psoCNN only have a maximum of 300 neurons in the fully-connected layers. In comparison with VGG16, which uses 4,096 neurons in its FC layers, this number is significantly smaller. Unfortunately, our available hardware would not allow us to search for more complex networks. Another issue with our limited computational power is that we could not run the algorithm with a larger number of particles for more iterations. More particles would help the algorithm explore more CNN architectures which could help to locate much better models for the problem at hand.

One major contribution to the field is the reduced running time of psoCNN. Although the stochastic nature of PSO and CNNs does not allow us to determine the algorithm's computational complexity analytically, our results show that psoCNN can find very competitive CNN architectures faster than evoCNN. Sun et al. showed that one single run of their evoCNN algorithm could take up to four days on the same datasets we tested here [19]. In that sense, the running time of the proposed algorithm was measured for 10 runs for the *MNIST* and *Convex* datasets with two different GPUs: a Nvidia P100 and a Nvidia GTX 960M. The former is a powerful GPU designed to be used in supercomputer clusters, while the latter one is merely a mobile GPU designed for use in laptop PCs. For the *MNIST* dataset, the best, average, and worst running time in the Nvidia P100 were 5.2, 3.33 and 7.3 hours, respectively. While in the Nvidia GTX 960M were 10.65, 15.22 and 21.64 hours, respectively. For the *Convex* dataset, the best, average, and worst running time in the Nvidia P100 were 0.411, 0.536, and 0.718 hours, respectively. While in the Nvidia GTX 960M were 1.36, 2.87, and 3.97 hours, respectively. We can see that even in a mobile GPU, the algorithm takes less than a day to process the *MNIST* dataset which is faster than evoCNN using two powerful Nvidia GTX 1080 GPUs. Furthermore, the running time of the proposed algorithm is compatible with the one reported by the authors of the IPPSO algorithm [29], but with significantly better results.

The best CNN architectures found by the proposed psoCNN is showed on Table 5. We can see that the algorithm prefers to use only a single fully-connected layer at the end of each CNN. This is important to notice because recent researches have confirmed that a single fully-connected layer at the end of a CNN produces better results than multiples fully-connected layers [49]. This shows that the proposed psoCNN algorithm is able to discover this structure without any domain knowledge about the problem.

Even though the algorithm is tested with small datasets, our results show that psoCNN is a *viable* solution to find optimized CNN architectures. Therefore, psoCNN can help deep learning practitioners to automatically design new CNN models for different domain problems.

6. Conclusion

In this work, we propose a novel algorithm to search for deep convolutional neural networks (CNNs) architectures based on particle swarm optimization (psoCNN). A novel directly encoding strategy is also proposed in which a CNN architecture is divided into two blocks: one block contains only convolutional and pooling layers, while the other contains only fully connected layers. This encoding strategy allows for variable length CNN architectures to be compared and combined using an almost standard PSO algorithm.

Our results show that psoCNN can quickly find an optimized CNN architecture for any given dataset. With only 30 particles and 20 iterations, the algorithm finds models capable of achieving test errors comparable to those designs exploiting more complex and complicated architectures. From our experimental results, we can conclude that psoCNN would be able to find even better architectures if more computational power is available.

For future works, we will expand the proposed psoCNN to use conflicting multiple objectives during the searching process, e.g., computational complexity and classification accuracy, in order to find multiple CNN models with different trade-offs between such objectives. In this regard, a decision maker would be able to select which model to use based on his or her needs. We will also exploit the CNN architectures

the algorithm will be able to find to include ResNets and DenseNets architectures as well, which will further increase the classification accuracy of the possible solutions.

Acknowledgments

This work is partially supported by the National Council for Scientific and Technological Development (CNPq, Brazil) grant 203076/2015-0. It used the Extreme Science and Engineering Discovery Environment (XSEDE), which is supported by the National Science Foundation (NSF, USA) grant number ACI-1548562, and the Bridges system, which is supported by National Science Foundation (NSF, USA) award number ACI-1445606, at the Pittsburgh Supercomputing Center (PSC). It also used the OSU High Performance Computing Center at Oklahoma State University supported in part through the National Science Foundation (NSF, USA) grant OCI–1126330.

Appendix A. Supplementary data

Supplementary data to this article can be found online at https://doi.org/10.1016/j.swevo.2019.05.010.

References

[1] K. He, X. Zhang, S. Ren, J. Sun, Delving deep into rectifiers: surpassing human-level performance on imagenet classification, in: 2015 IEEE International Conference on Computer Vision (ICCV), IEEE, 2015, pp. 1026–1034, https://doi.org/10.1109/ICCV.2015.123.

[2] S. Ioffe, C. Szegedy, Batch normalization: accelerating deep network training by reducing internal covariate shift, in: International Conference on Machine Learning, 2015, pp. 448–456.

[3] K. Simonyan, A. Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, 2014. arXiv preprint arXiv:1409.1556.

[4] C. Szegedy, Wei Liu, Yangqing Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, Boston, MA, USA, 2015, pp. 1–9, https://doi.org/10.1109/CVPR.2015.7298594.

[5] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, Las Vegas, NV, USA, 2016, pp. 770–778, https://doi.org/10.1109/CVPR.2016.90.

[6] G. Huang, Z. Liu, L.V.D. Maaten, K.Q. Weinberger, Densely connected convolutional networks, in: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, Honolulu, HI, 2017, pp. 2261–2269, https://doi.org/10.1109/CVPR.2017.243.

[7] M. Lin, Q. Chen, S. Yan, Network in Network, 2013. arXiv preprint arXiv:1312.4400.

[8] M. Schoenauer, E. Ronald, Neuro-genetic truck backer-upper controller, in: Proceedings of the First IEEE Conference on Evolutionary Computation. IEEE World Congress on Computational Intelligence, 1994, pp. 720–723, https://doi.org/10.1109/ICEC.1994.349969.

[9] E. Ronald, M. Schoenauer, Genetic lander: an experiment in accurate neuro-genetic control, in: International Conference on Parallel Problem Solving from Nature, Springer, 1994, pp. 452–461.

[10] J. Schaffer, R.A. Caruana, L.J. Eshelman, Using genetic search to exploit the emergent behavior of neural networks, Phys. Nonlinear Phenom. 42 (13) (1990) 244–248, https://doi.org/10.1016/0167-2789(90)90078-4.

[11] H. Kitano, Empirical studies on the speed of convergence of neural network training using genetic algorithms, in: Proceedings of the Eighth National Conference on Artificial Intelligence, vol. 2, AAAI Press, 1990, pp. 789–795. AAAI90.

[12] K.O. Stanley, R. Miikkulainen, Evolving neural networks through augmenting Topologies, Evol. Comput. 10 (2) (2002) 99–127, https://doi.org/10.1162/106365602320169811.

[13] N.T. Siebel, G. Sommer, Evolutionary reinforcement learning of artificial neural networks, Int. J. Hybrid Intell. Syst. 4 (3) (2007) 171–183, https://doi.org/10.3233/HIS-2007-4304.

[14] K.O. Stanley, D.B. D'Ambrosio, J. Gauci, A hypercube-based encoding for evolving large-scale neural networks, Artif. Life 15 (2) (2009) 185–212, https://doi.org/10.1162/artl.2009.15.2.15202.

[15] P. Verbancsics, J. Harguess, Generative Neuroevolution for Deep Learning, 2013. arXiv preprint arXiv:1312.5355.

[16] E. Real, S. Moore, A. Selle, S. Saxena, Y.L. Suematsu, J. Tan, Q.V. Le, A. Kurakin, Large-scale evolution of image classifiers, in: D. Precup, Y.W. Teh (Eds.), Proceedings of the 34th International Conference on Machine Learning, Vol. 70 of Proceedings of Machine Learning Research, PMLR, International Convention Centre, Sydney, Australia, 2017, pp. 2902–2911.

[17] H. Liu, K. Simonyan, O. Vinyals, C. Fernando, K. Kavukcuoglu, Hierarchical Representations for Efficient Architecture Search, 2017. arXiv preprint arXiv:1711.00436.

[18] H. Jin, Q. Song, X. Hu, Efficient Neural Architecture Search with Network Morphism, 2018. arXiv preprint arXiv:1806.10282.

[19] Y. Sun, B. Xue, M. Zhang, G.G. Yen, Evolving deep convolutional neural networks for image classification, IEEE Trans. Evol. Comput. (2018), https://doi.org/10.1109/TEVC.2019.2916183.

[20] Y. Sun, G.G. Yen, Z. Yi, Evolving unsupervised deep neural networks for learning meaningful representations, IEEE Trans. Evol. Comput. 23 (1) (2019) 89–103, https://doi.org/10.1109/TEVC.2018.2808689.

[21] V. Gudise, G. Venayagamoorthy, Comparison of particle swarm optimization and backpropagation as training algorithms for neural networks, in: Proceedings of the 2003 IEEE Swarm Intelligence Symposium. SIS03 (Cat. No.03EX706), vol. 2, IEEE, 2003, pp. 110–117, https://doi.org/10.1109/SIS.2003.1202255.

[22] M. Carvalho, T.B. Ludermir, Particle swarm optimization of feed-forward neural networks with weight decay, in: 2006 Sixth International Conference on Hybrid Intelligent Systems (HIS06), IEEE, 2006, 55.

[23] M. Carvalho, T.B. Ludermir, Particle swarm optimization of neural network architectures and weights, in: 7th International Conference on Hybrid Intelligent Systems (HIS 2007), IEEE, 2007, pp. 336–339, https://doi.org/10.1109/HIS.2007.45.

[24] S. Kiranyaz, T. Ince, A. Yildirim, M. Gabbouj, Evolutionary artificial neural networks by multi-dimensional particle swarm optimization, Neural Netw. 22 (10) (2009) 1448–1462, https://doi.org/10.1016/j.neunet.2009.05.013.

[25] J.-R. Zhang, J. Zhang, T.-M. Lok, M.R. Lyu, A hybrid particle swarm optimizationback-propagation algorithm for feedforward neural network training, Appl. Math. Comput. 185 (2) (2007) 1026–1037, https://doi.org/10.1016/j.amc.2006.07.025.

[26] B. Qolomany, M. Maabreh, A. Al-Fuqaha, A. Gupta, D. Benhaddou, Parameters optimization of deep learning models using Particle swarm optimization, in: 2017 13th International Wireless Communications and Mobile Computing Conference (IWCMC), IEEE, 2017, pp. 1285–1290, https://doi.org/10.1109/IWCMC.2017.7986470.

[27] A. Kenny, X. Li, A study on pre-training deep neural networks using particle swarm optimisation, in: Asia-Pacific Conference on Simulated Evolution and Learning, Springer, 2017, pp. 361–372.

[28] Y. Sun, B. Xue, M. Zhang, G.G. Yen, A particle swarm optimization-based flexible convolutional autoencoder for image classification, IEEE Trans. Neural Netw. Learn. Syst. (2018) 1–15, https://doi.org/10.1109/TNNLS.2018.2881143.

[29] B. Wang, Y. Sun, B. Xue, M. Zhang, Evolving deep convolutional neural networks by variable-length particle swarm optimization for image classification, in: 2018 IEEE Congress on Evolutionary Computation (CEC), 2018, pp. 1514–1521.

[30] J. Kennedy, R.C. Eberhart, Swarm Intelligence, Academic Press, San Diego, CA, 2001.

[31] A. Sahu, S.K. Panigrahi, S. Pattnaik, Fast convergence particle swarm optimization for functions optimization, Procedia Technol. 4 (2012) 319–324, https://doi.org/10.1016/j.protcy.2012.05.048.

[32] Hu Wang, G.G. Yen, Adaptive multiobjective particle swarm optimization based on parallel cell coordinate system, IEEE Trans. Evol. Comput. 19 (1) (2015) 1–18, https://doi.org/10.1109/TEVC.2013.2296151.

[33] Y. Lecun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, Proc. IEEE 86 (11) (1998) 2278–2324, https://doi.org/10.1109/5.726791.

[34] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780, https://doi.org/10.1162/neco.1997.9.8.1735.

[35] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, Learning phrase representations using RNN encoderdecoder for statistical machine translation, in: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, Stroudsburg, PA, USA, 2014, pp. 1724–1734, https://doi.org/10.3115/v1/D14-1179.

[36] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016, http://www.deeplearningbook.org.

[37] D.P. Kingma, J. Ba, Adam: A Method for Stochastic Optimization, 2014. arXiv preprint arXiv:1412.6980.

[38] X. Glorot, Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, in: Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, 2010, pp. 249–256.

[39] S. Ioffe, C. Szegedy, Batch normalization: accelerating deep network training by reducing internal covariate shift, in: Proceedings of the 32nd International Conference on International Conference on Machine Learning, vol. 37, JMLR. org, 2015, pp. 448–456.

[40] Y. LeCun, B. Boser, J.S. Denker, D. Henderson, R.E. Howard, W. Hubbard, L.D. Jackel, Backpropagation applied to handwritten zip code recognition, Neural Comput. 1 (4) (1989) 541–551, https://doi.org/10.1162/neco.1989.1.4.541.

[41] H. Larochelle, D. Erhan, A. Courville, J. Bergstra, Y. Bengio, An empirical evaluation of deep architectures on problems with many factors of variation, in: Proceedings of the 24th International Conference on Machine Learning, ACM, 2007, pp. 473–480.

[42] H. Xiao, K. Rasul, R. Vollgraf, Fashion-mnist: a Novel Image Dataset for Benchmarking Machine Learning Algorithms, 2017. arXiv preprint arXiv:1708.07747.

[43] M. Liang, X. Hu, Recurrent convolutional neural network for object recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 3367–3375.

[44] C.-Y. Lee, P. Gallagher, Z. Tu, Regularization of neural networks using DropConnect, in: International Conference on Machine Learning 2013, 2013, pp. 2095–2103.
[45] S. Rifai, P. Vincent, X. Muller, X. Glorot, Y. Bengio, Contractive auto-encoders: explicit invariance during feature extraction, in: Proceedings of the 28th International Conference on International Conference on Machine Learning, ICML11, Omnipress, USA, 2011, pp. 833–840.
[46] T.-H. Chan, K. Jia, S. Gao, J. Lu, Z. Zeng, Y. Ma, PCANet: a simple deep learning baseline for image classification? IEEE Trans. Image Process. 24 (12) (2015) 5017–5032, https://doi.org/10.1109/TIP.2015.2475625.
[47] R. Benenson, Classification datasets results, http://rodrigob.github.io/are_we_there_yet/build/classification_datasets_results.html, accessed on September 06, 2018.
[48] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, in: Advances in Neural Information Processing Systems, 2012, pp. 1097–1105.
[49] J.T. Springenberg, A. Dosovitskiy, T. Brox, M. Riedmiller, Striving for Simplicity: the All Convolutional Net, 2014. arXiv preprint arXiv:1412.6806.